Hashing Algorithms

1

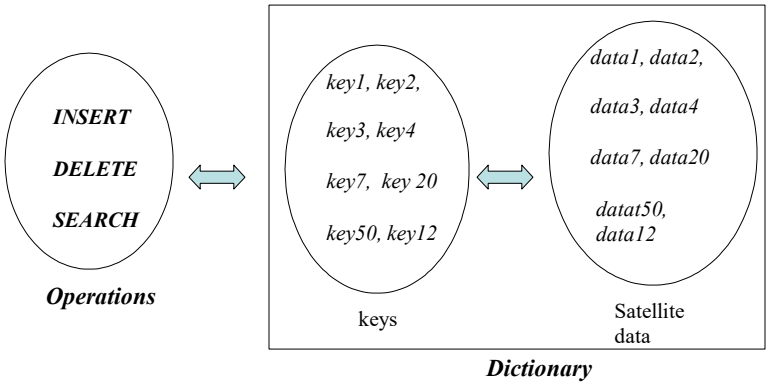
Hashing

Dictionary

The ***hashing algorithms*** are often used on a special data structure called ***dictionary.***

- A dictionary is a dynamic data structure consisting of a ***set of keys.*** It supports three basic operations: ***insertion***, ***deletion,*** and ***search.***

- Generally, the keys in a dictionary can have additional related elements, called ***satellite data,*** as illustrated in the diagram .



2

Dictionary

Examples

Many real life applications use dictionaries, consisting of keys based on ***numbers and/or alphabets.***

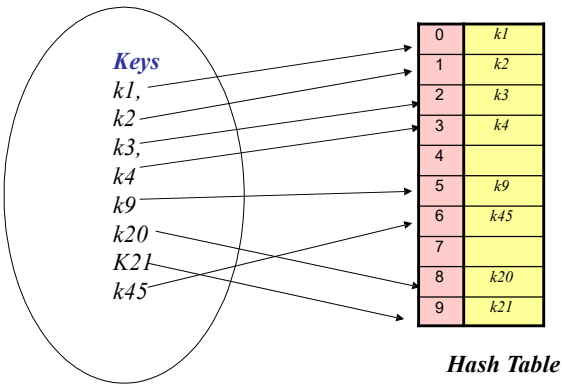
- *Set of* ***Personnel Numbers*** *{13456, 7890, 2348, 1256… }*
- *Set of* ***Part numbers*** *(111223-5, 67890-6, 2345-8, 789011-29, ..}*
- ***Symbol Table*** *used by a compiler*
- ***Online dictionary*** *for spell checking*

3

Hashing Algorithms

Hash Table

Hashing is the procedure of mapping dictionary keys into a set of *m* integers in range *0, 1,……. m-1***.** The mapped keys are stored into table called ***hash table*** The table consists of *m cells*.



4

Hashing

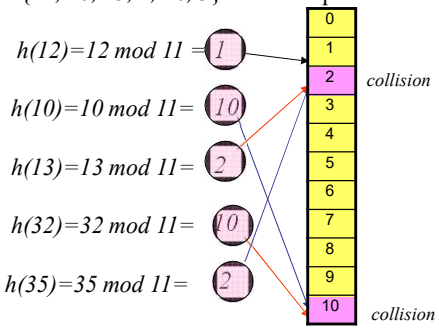
Collisions

A hash function generally ***maps multiple keys*** into the ***same integer value.*** The keys being mapped are said to have ***collisions***, because they all belong to ***same slot*** in the hash table.

- Consider the hash function :

$$h(k) = k \bmod 11$$

The keys {*12, 10, 13, 2, 14, 3*} would map as follows



In this example, the keys {*10 , 32*} map to cell 10, and the keys { *13, 35*} map to cell 2. Thus, the keys *10* and *32* have ***collisions***. Same way, the keys *13* and *35* have collisions.

5

Hashing Algorithm

Collision Resolution

Two basic approaches to collision resolution are called ***chained hashing*** and ***open address hashing***

Chained Hashing: In chained hashing the elements of a hash table are stored in a set of ***linked lists.***

- *All colliding elements are kept in **one linked list.***
- *The list head pointers are usually stored in an array.*
- *Chained hashing is also known as **open hashing***

Open Address Hashing: In open address hashing the hashed keys are stored in the ***hash table*** itself.

- *The colliding keys are allocated distinct cells in the table.*
- *Open address hashing is also referred to as **closed hashing***

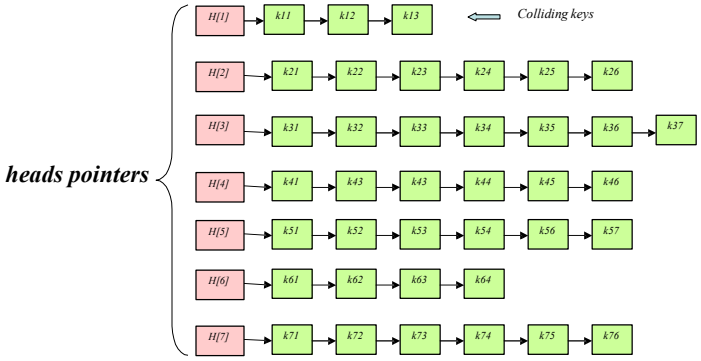
6

Chained Hashing

Structure

In ***chained hashing*** a ***set of linked lists*** is used to store keys. The colliding keys are maintained in the same linked list. The number of linked lists equals the size of hash table . An ***array of pointers*** provides entry points to the linked lists.

- The figure illustrates open hashing representation of hash table The linked lists pointers are *H[1],H[2].H[7].* The keys *k11,k12,* and *k13*, for example, have collisions, and are stored in the first list.



- The chained hashing is ***flexible***. The new keys can be inserted easily. Searching is fast. However, there is ***storage overhead*** for pointers. Also, deletion is inefficient.

7

Hashing

Modulo Function

Several functions can be used to map keys into a set of integers. The choice is made on the basis of amount of computation time required, and simplicity of the computational steps. A common choice is a ***modulo function*** *h(x)* defined as:

$$h(k) = k \bmod m$$

where ***k*** is the ***key*** , ***m*** is some positive integer and ***mod*** denotes the modulus operator which computes the remainder of key *k* divided by *m*.

- It follows that the hash function *h(x)* maps the set of keys $\{k_1, k_2, k_3, \ldots\ldots k_n\}$ into a set of integers $\{0,1,2,\ldots\ldots m-1\}$

- In essence, the modulo function is used to create a hash table of size *m*

8

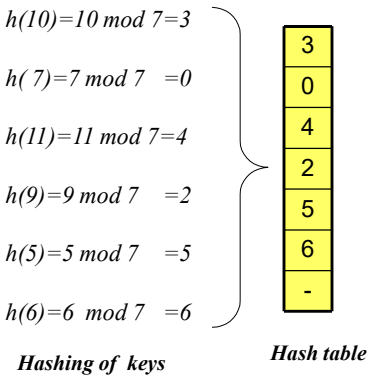
Hashing

Modulo Function

Consider the modulo hash function

$$h(k) = k \bmod m$$

▪If m is taken, for example, as 7, the hash table would contain up to seven entries. Further, in this case, the set of keys $\{10, 7, 11, 9, 5,6\}$ would map as follows:



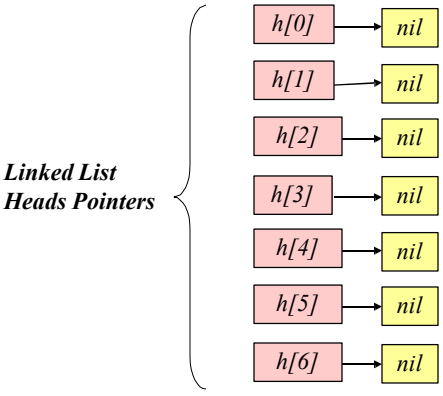
Hashing of keys ***Hash table***

9

Chained Hashing

Insertions

Hashing function: h(k)= k mod m, m = 7



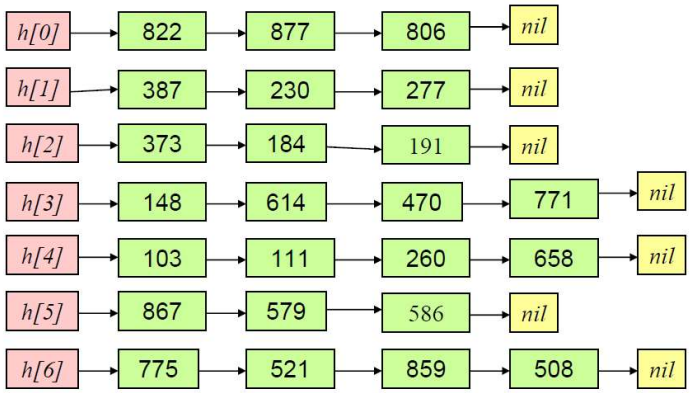
Empty table

10

Chained Hashing

Insertions

- The figure shows the chained hash table after insertion of keys {822,387,373,143,103,867,775,877,230,184,614,111,579,521,806,277,191,470,260,586,859,771,658,508}



Chained hash table

11

Analysis of Chained Hashing

Models

For the purpose of analysis, two simplified model of chained hashing are considered.

- In one model of hashing it is assumed that all hash keys are ***mapped to a single list***. This situation may referred to as worst distribution of hash keys. In practice, this extreme situation may not arise, but nevertheless, possibility does exit.

- In a second model, it is assumed that the keys are ***uniformly distributed among all the linked lists***. This too is an idealized situation. However, the assumption can be used to work out the running costs of basic operations of ***search*** and ***insertion***.

- The search can be ***successful*** or ***unsuccessful***. The unsuccessful search basically involves the same steps as those in insertion operation. In both cases, a linked list is probed up to the tail node. In the case of insertion this necessary in order to ensure the key to be inserted is not a duplicate.

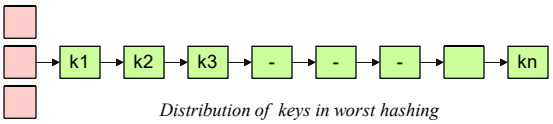
- I n search and insertion operations, it is usual to express the running cost in terms of number of ***probes*** required to search or insert an element.

12

Analysis of Chained Hashing

Worst Hashing :Search

The distribution of keys in ***worst hashing*** is shown in the figure. All keys are stored in a single list. The search can be ***successful*** or ***unsuccessful.*** The analysis is simple, since it boils down to examining a single ***linear list***. ..



Distribution of keys in worst hashing

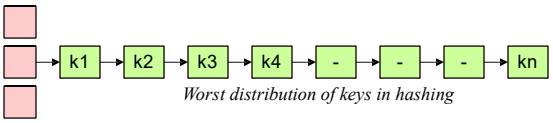
- First we consider the ***successful search.***
  - The best search time is $\theta(1)$, since the key will be will be stored in the front node.
  - In worst case, the key will be found in the last node. Thus, worst case search time $\theta(n)$
  - On an average the list will be examined $n/2$ times. Thus, average search time is $\theta(n/2) = \theta(n)$
- In case of unsuccessful search all of the n stored keys are examined . Therefore, the running time for ***unsuccessful search*** is $\theta(n)$

13

Analysis of Chained Hashing

Worst Hashing: Insertions

The worst situation arises when ***all keys hash to same linked list,*** as depicted in the figure.



Worst distribution of keys in hashing

- A ***key*** is inserted at the end of the list. It requires ***n probes to detect any duplicate keys***. Thus, insertion time is $\theta(n)$
- Next we examine the case of ***inserting n keys***. Let $c1$ be the cost of determining the pointer to linked list and $c2$ the cost of probing a node. When first key is inserted, no node is probed. Thus, cost of insertion of first key is $c1$. When 2nd key is inserted, only one node is probed. Therefore, the cost of insertion of 2nd key is $c1+c2$. In general, when k^{th} key is inserted the cost is $c1+(k-1)c2$. The table below summarizes the insertion steps and associated costs.

# Insert key	***# of nodes probed***	***cost***
Ist	*0*	*c1*
2nd	*1*	*c1+c2*
3rd	*2*	*c1+2.c2*
----	----	------
k^{th}	*k-1*	*c1+(k-1)c2*
-----	-----	----
n^{th}	*n-1*	*c1+(n-1)c2*

- The average cost of ***inserting a key into*** a single list is given by:

$I = n.c1 + c2(1+2+3+\ldots\ldots+(n-1)) = n.c1 + c2\, n(n-1)/2$

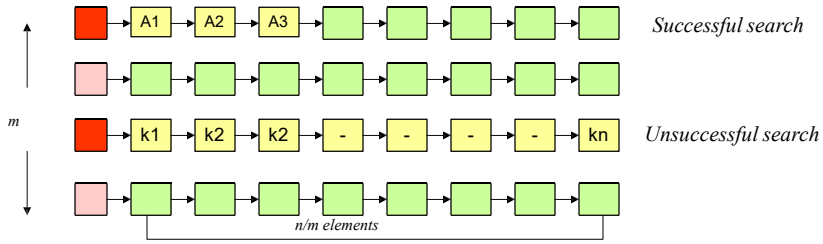
$I_{avg} = \frac{1}{n}\left(\frac{n.c1 + c2\, n(n-1)}{2}\right) = \theta(n)$

14

Analysis of Chained Hashing

Uniform Hashing :Search

We assume that all hashed keys are ***uniformly distributed*** over the linked lists. It means that each linked list has nearly the same number of keys, as shown in the figure.



Chained hashing with uniform distribution of keys

▪Let *m* be the ***table size***. Therefore, there are *m* linked lists in the chained hashing. If *n* is the total number keys in the table, then ***average size*** of each linked list is *n/m*. The ratio α=*n/m* is called ***load factor***. We consider cases of successful and unsuccessful searches.

▪To search a key, first the pointer to the appropriate list is determined. This operation has ***fixed*** cost. Let c_1 be cost. In unsuccessful search, the list is probed up to the end Thus, in case of unsuccessful search, *n/m* nodes are examined. If c_2 is unit cost of probing a node, the total time for ***unsuccessful search*** is given by

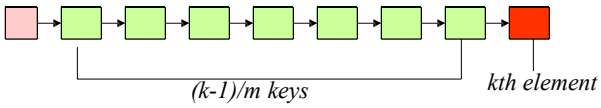
$$T(n) = c_1 + c_2.(n/m) = \theta(1+n/m) = \theta(1+\alpha)$$

15

Analysis of Chained Hashing

Uniform Hashing : Insertions

In order to find average time for inserting a key let us consider the case when *kth* key is inserted. At that stage, the list has already *k-1* keys ***distributed uniformly over m linked lists***. Thus, prior to insertion of *kth* key, the average length of each list is *(k-1)/m*, as shown in the diagram.



The insertion of new key would require probing of ***(k-1)/m keys*** plus the cost of adding new key. Thus, the overall cost of insertion of *kth* key is *1+(k-1)/m, assuming that each operation consumes unit time 1*. The expected cost of inserting a key is obtained by summing over all possible values of *k*. Thus, the ***expected cost I*** is given by:

$$I = \frac{1}{n}\sum_{k=1}^{n}\left(1+\frac{k-1}{m}\right)$$

$$= 1 + 1/n\left(\sum_{k=1}^{n} k/m - \sum_{k=1}^{n} 1/m\right) = n(n-1)/2m - n/m = 1 + n/2m - 1/2m$$

$$= 1 + \alpha/2 - 1/2m$$

The ***average cost for inserting a key is*** $\theta(1 + \alpha/2 - 1/2m) = \theta(1+\alpha)$

16

Open Address Hashing

Linear Probing

In ***linear probing*** the hashed key is incremented by an integer value In general the hash function is defined as function

$$h(k,i)=(h'(k)+ i\ ) \bmod m,$$

where $h'(k)$ is an ***auxiliary hash function*** and m is the table size.

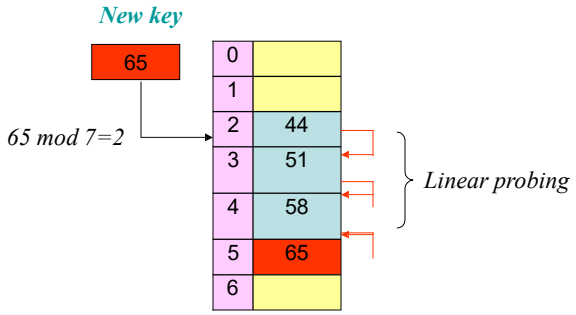
▪Linear function is easy to compute. The auxiliary function may be chosen as mod or some other hash function .

17

Open Address Hashing

Linear Probing-1

$$h(k,i) = ((k \bmod m) + i) \bmod m, \quad \text{where} \quad i=0,1,2,3..m-1$$



i	*k mod m + i*	*h(k,i)*	*Slot Status*
0	*2+0=2*	*2*	*Slot 2 occupied by 44*
1	*2+1=3*	*3*	*Slot 3 occupied by 51*
2	*2+2=4*	*4*	*Slot 4 occupied by 58*
3	*2+3=5*	*5*	*Slot 5 empty, inserted 64*

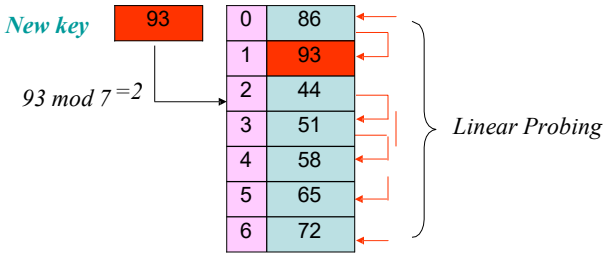
Linear probing status

18

Open Address Hashing

Linear Probing-2

$h(k,i)=((k \mod m)+ i) \mod m, \quad$ where $i=0,1,2,3..m-1$



i	*k mod m + i*	*h(k,i)*	*Slot Status*
0	*2+0=2*	*2*	*Slot 2 occupied by 44*
1	*2+1=3*	*3*	*Slot 3 occupied by 51*
2	*2+2=4*	*4*	*Slot 4 occupied by 58*
3	*2+3=5*	*5*	*Slot 5 occupied by 65*
4	*2+4=6*	*6*	*Slot 6 occupied by 72*
5	*2+5=7*	*0*	*Slot 0 occupied by 86*
6	*2+6=8*	*1*	*Slot 1 empty is empty, inserted 93*

Linear probing status

19

Linear Probing

Insertion

The ***HASH-INSERT*** method inserts a key into the hash table using the following linear probing function:

$h(k,i)=((k \mod m)+ i) \mod m$

```
HASH-INSERT(T, k)
1   i ← 0
2   do while i < m
4       j ←( (k mod m )+ i ) mod m
5       if T[j]=NIL
6           then T[j]←k
7                return j
8       else i ← i+1
9   Error "Table full "
```

20

Linear Probing

SEARCH

The ***HASH-SEARCH*** method searches a key in the hash table . The table is constructed using the following linear probing function:

$$h(k,i)=((k \mod m)+ i) \mod m$$

```
HASH-SEARCH(T, k)
1    i ← 0
2   do while i< m
4       j ←( (k mod m )+ i ) mod m
5       if T[j]=k
6           return j
7       else  i ← i+1
8  return -1
```

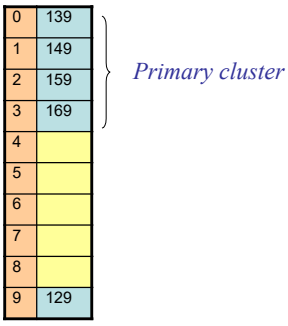
21

Linear Probing

Limitation

The ***linear probing*** can be easily implemented**.** However, it has the some disadvantages. Long hashing runs result in grouping of keys in consecutive slots. The grouping is called ***primary clustering*** . Clustering slows down the search operation

▪Suppose, the hash table has size is *10*. If in hashing, the linear probe is used. It can be seen that , the set of keys *{ 129,139, 149,159}* will hash to consecutive slots, forming a cluster. The behavior is illustrated in the picture.



Hashing with linear probe

22

Open Address Hashing

Quadratic Probing

In ***quadratic probing*** the hashed key is incremented by square of an integer value
In general the hash function is defined as function

$$h(k,i)=(h'(k)+ c_1 i+c_2 i^2) \bmod m,$$

where $h'(k)$ is an ***auxiliary hash function*** and m is the table size. c_1 and c_2 are integers. For example, c_1 may be chosen as 0 and c_2 as 1

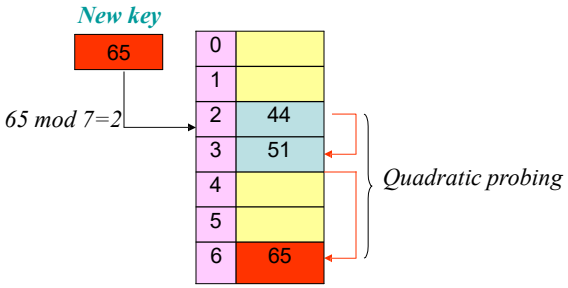
▪Quadratic probing minimizes the problem of primary clustering, because hashed keys are mapped to distant slots

23

Open Address Hashing

Quadratic Probing-1

$$h(k,i) = ((k \bmod m) + i^2)) \bmod m, \quad \text{where} \quad i=0,1,2,3..m-1$$



i	i^2	*k mod m* $+i^2$	*h(k,i)*	*Slot Status*
0	*0*	*2+0=2*	*2*	*Slot 2 occupied by 44*
1	*1*	*2+1=3*	*3*	*Slot 3 occupied by 51*
2	*4*	*2+4=6*	*6*	*Slot 6 empty, 65 inserted*

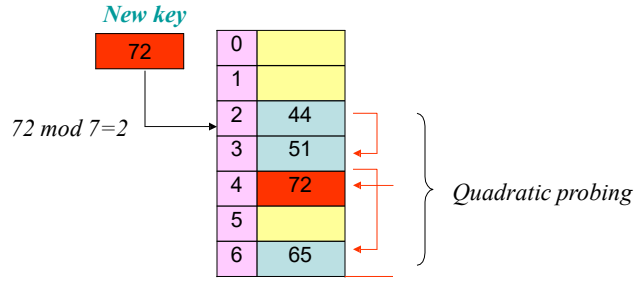
Quadratic probing status

24

Open Address Hashing

Quadratic Probing-2

$h(k,i) = ((k \bmod m) + i^2)) \bmod m, \quad$ where $i=0,1,2,3..m-1$



i	i^2	*k mod m* $+i^2$	*h(k,i)*	*Slot Status*
0	*0*	*2+0=2*	*2*	*Slot 2 occupied by 44*
1	*1*	*2+1=3*	*3*	*Slot 3 occupied by 51*
2	*4*	*2+4=6*	*6*	*Slot 6 occupied by 65*
3	*9*	*2+9=11*	*4*	*Slot 4 is empty, 72 inserted*

Quadratic probing status

25

Quadratic Probing

Insertion

The ***HASH-INSERT*** method inserts a key into the hash table using the following quadratic probing function:

$h(k,i)=((k \bmod m)+i^2) \bmod m$

```
HASH-INSERT(T, k)
1    i ← 0
2   do while i < m
4       j ←( (k mod m )+ i²  ) mod m
5       if T[j]=NIL
6           then   T[j]←k
7                  return j
8       else   i ← i+1
9  Error  "Table full "
```

26

Quadratic Probing

SEARCH

The ***HASH-SEARCH*** method searches a key in the hash table . The table is constructed using the following quadratic probing function:

$$h(k,i)=((k \mod m)+ i^2) \mod m$$

```
HASH-SEARCH(T, k)
1    i ← 0
2   do while i< m
4       j ←( (k mod m )+ i² ) mod m
5       if T[j]=k
6           return j
7       else  i ← i+1
8  return -1
```

27

Open Address Hashing

Double Hashing

The double hashing uses two auxiliary function to has the keys. A typical set of functions is as follows

$$h(k, j)= (h_1(k) + j\, h_2(k)) \mod m$$

where h_1 and h_2 are auxiliary hash functions.

For example, the auxiliary function might be chosen as

$h_1(k)= k \mod m$
$h_2(k) = 1+ (k \mod p)$
where $p < m$

- The double hashing strategy reduces the chances of primary and secondary clustering. However, it involves additional computations.

28

Analysis of Open Address Hashing

Insertions

$$I = O\left(\frac{1}{1-\alpha}\right)$$

- A key is inserted into the table after one or more probes are made to find a vacant slot. Thus, the cost of inserting a key may interpreted as the ***average number of probes required to find a slot in the hash table .***

If table is *three quarter full*, i.e, α=3/4 =0.75,I= 1/(1-0.75)=1/0.25=4. Thus, on an average four probes would be required to insert key.

If table is 90 % full, I=1/(1-.0.9)=10 i.e on an average 10 probes are required to insert a key.

29

Analysis of Open Address Hashing

Unsuccessful Search

An ***unsuccessful search*** occurs when a given key is not found in the hash table.

- The case of ***unsuccessful search*** is identical to that of inserting a new key into the hash table. For, to insert a new key we need to probe if a slot is available and that the key is not a duplicate. The last condition implies that search for duplicate key is unsuccessful. Or, in other words, the search for finding the new key is unsuccessful.

- Thus, the average number of probes for unsuccessful search is equal to average number of probes for inserting a key.

- Using the preceding analysis, it is concluded that ***average number of probes, U, for unsuccessful search*** is given by

$$U = 1/(1-\alpha)$$

30

Open Address Hashing

Rehashing

In ***open address hashing***, the size of hash table should be greater than or equal to the number of keys to be stored in the table.

▪In some applications, the number of keys is not in advance. In such situation, a table of some estimated size is created initially. When the number of input keys exceeds the hash table size, a new table of larger size is created, which may twice as large as the original table. All the keys are transferred to the new table. This process is known as ***rehashing***.

▪In rehashing the keys in the original table are ***not simply copied*** to the new table. Instead, all elements are rehashed using the new table size. Assuming that the original table was of size *m* and new table twice as large, then hash function for the old and new tables might be as follows:

$h_{old}(k) = k \bmod m$

$h_{new}(k) = k \bmod 2m$

.

31